

Measuring Long-Horizon Agentic Capability: Duration, Economic Value, and Synthesis Quality

How METR measures task duration, GDPval measures real-world economic value against experts, and DeepScholar-Bench measures whether models can actually do research synthesis well

Why Agentic Evaluation Is Getting Harder

Measuring AI progress means measuring what actually matters. A few years ago, just getting a model to produce a coherent explanation was a big deal. Now, the more meaningful questions are about real-world impact: how these models affect economics, safety, and what tasks they can actually be trusted to do.

Traditional benchmarks — single-turn chatbot Q&A, answering something already sitting in the context — are **saturating quickly**. To meaningfully track progress now, two different questions matter more:

1. **How long or complex a task can a model actually complete?**
2. **What's the real-world economic value of tasks a model can do?**

Two benchmarks map onto these two questions:

- **METR's time-horizon metric** — calibrated against how long a human professional would take to do the same task.
- **GDPVal** — compares a model's win rate on a task directly against human experts.

Both are valid, complementary ways of measuring progress, even though the trends they reveal can look quite different.

METR: Measuring AI Ability to Complete Long Tasks

Paper: arxiv.org/abs/2503.14499

A few years ago, chatbot conversations would fall apart after just a couple of turns — the model would lose track of context. Today, conversations can run much longer. This points to a core capability question: **how long a task can a model actually complete?**

METR's approach measures two things together:

1. **Duration:** how long would this task take a skilled human professional?

- 2. **Reliability:** what fraction of the time does the model actually succeed at it (50%? 80%?)?

The **time horizon** metric is the human-professional-equivalent duration of the hardest tasks a model can complete at a given success rate — using human task-completion time as the universal, comparable anchor across every kind of task.

The Three Task Suites

METR's benchmark spans about **170 tasks total**, ranging from seconds to hours:

Suite	Task length	Focus	# of tasks
SWAA	1–30 seconds	Atomic actions (e.g., "open a file")	66
HCAST	1 minute – 30 hours	Diverse software/research engineering tasks	97
RE-Bench	Up to ~8 hours	Full ML research-style tasks	7

Example tasks from the benchmark: finding a shell script (~3 seconds), a Wikipedia research task, detecting and fixing a bug in simulation input files (~10 minutes for a human), transforming JSON data from one format to another, and building a CUDA kernel backtesting tool aiming for a specific performance improvement (~8 hours for a human).

How the Time Horizon Is Actually Measured

- 1. **Collect and vet tasks** across the three suites.
- 2. **Time skilled humans** (roughly 5 years of professional experience in the relevant field) on successful attempts, then take the **geometric mean** of their completion times as the task's difficulty rating.
- 3. **Run the same tasks through models**, measuring whether each one succeeds.
- 4. **Fit a curve** relating success rate to human-equivalent task duration — this curve is what produces the "time horizon" for a given model at a given reliability level (e.g., 50% or 80%).

A Known Bias in the Human Baseline

Experienced professionals are often conditioned to what "success" looks like in their own field, so they can tend to **underestimate** how hard a task actually is — which may not match how difficult a model finds the same task. This is a known limitation of using expert time estimates as the anchor.

Rating Agreement

Since getting many experts to solve every task is expensive, the typical approach uses a **small number of expert solvers** and checks their inter-rater agreement — if their times disagree substantially, more data points are collected; if they broadly agree, the estimate is used as-is.

Success Rate vs. Task Length

Plotting model success rate (0 to 1) against human time-to-complete:

- **SWAA tasks (1–30 seconds):** success rates are generally high.
- **HCAST tasks (minutes to hours):** results are much more scattered — anywhere from ~0.8 down to 0.
- **RE-Bench tasks (up to ~8 hours):** success rates tend to be **lower** overall at this point, though some newer work reports progress here.

The clear pattern: the longer the task, the harder it is for a model to stay on track and complete it reliably — which is exactly why measuring "how long a task can this model handle" has become such an important axis for agentic evaluation.

The Headline Trend: Time Horizon Is Doubling

Plotting a model's 50%-success time horizon against its release date:

- **GPT-2 (2019):** about 2 seconds.
- **GPT-4 (2023):** a few minutes.
- **Claude 3.7 Sonnet and o1 (2025):** roughly an hour (reported figures vary somewhat by exact model/version, generally in the 50–60 minute range).

This trend has been **doubling roughly every 7 months**. But there's an important caveat: this is measured at only **50% reliability** — meaning the model succeeds only about half the time at that task length. That's roughly equivalent to handing a task to an intern who completes it correctly only half the time; there's real, persistent uncertainty about whether any individual attempt will actually succeed.

What's Actually Driving This Improvement?

Several capabilities compound to drive the time-horizon trend upward:

- **Better logical reasoning** — directly connects to reasoning-model research covered elsewhere.
- **Improved code generation** — a recurring theme across coding-focused research.
- **Better tool use** — connects to ReAct-style interleaved reasoning and acting.

- **Improved reliability** — models are less likely to get stuck repeating the same unproductive behavior.
- **Better error recovery** — on a long task, some steps will inevitably go wrong; recovering requires the model to track its overall goal and its progress toward it (a form of state/memory), not just react to the most recent step in isolation.

Additional Contributing Factors

- **Context engineering:** better structuring of what goes into the context window over long, multi-step sessions (e.g., how context gets compacted as a session approaches its length limit) meaningfully improves practical performance.
- **Learning from real usage patterns:** products can incorporate feedback from how people actually use them (e.g., what gets accepted vs. rejected) to improve behavior over time on the classes of problems that come up most often.
- **Explicit planning on complex problems:** when a problem is complex enough, an agent may first generate an explicit plan, breaking the task into steps — and when execution reveals something unexpected, it can **re-plan** rather than blindly continuing the original steps.
- **User feedback loops:** a user pointing out that generated code is wrong and needs to be redone is itself a form of feedback the model has to act on mid-task.
- **Memory of the environment:** understanding of a specific codebase or environment, retained across a session, helps the model calibrate its expectations and avoid re-learning the same context repeatedly.

Reliability Is a Separate, Harder Axis

Plotting time horizon by release date again, but this time comparing the **50% success** curve against the **80% success** curve: while the 50%-reliability time horizon reached up to roughly an hour by 2025, the **80%-reliability** time horizon is still much shorter — closer to **8–15 minutes** for the same top models (for example, Claude 3.7 Sonnet's 80% time horizon was about 15 minutes, versus roughly an hour at 50% reliability).

The takeaway: there's a big, persistent gap between "can technically do this task sometimes" and "can be trusted to do this task reliably." That gap represents real headroom — and real risk — for deploying these models on moderately complex real-world tasks, since high-reliability performance still lags well behind the more attention-grabbing 50%-reliability headline numbers.

Common Failure Modes

Comparing failure patterns between GPT-4 (non-reasoning) and o1 (reasoning) on these long-horizon tasks:

- **Poor planning** — the model doesn't break the task down into sensible steps.

- **Poor tool choice** — picks the wrong tool for a given step.
- **Incorrect reasoning/mental math** — makes a calculation or logical error mid-task.
- **Premature abandonment** — gives up or loses track of what "success" actually means for the task, without a repetitive-loop trigger.
- **Repetitive loops** — the model keeps repeating the same (still highest-probability) action even after it has already failed, rather than trying something different. This failure mode was notably common in GPT-4, and less so in o1.

Understanding these specific failure patterns is directly useful for knowing where a system needs to improve — a common evaluation approach is to have a person read through model transcripts directly to identify recurring failure patterns, rather than relying purely on aggregate pass/fail numbers.

Known Limitations of This Benchmark

No benchmark is perfect, and METR's approach has some specific, acknowledged limitations:

- **Weaker on "messy" tasks** — tasks without one single correct answer, or with real ambiguity, tend to show lower model performance; the same doubling-time trend likely still holds within different messiness categories, but it's less cleanly measured.
- **SWE-bench-specific bias** — SWE-bench shows a similar overall trend, but human annotators tend to **underestimate** task difficulty there, partly because models have likely already seen many of the underlying GitHub repositories during training — which tends to shorten the *apparent* doubling time relative to a genuinely unseen codebase.
- **Contractor-like performance on unfamiliar codebases** — on internal, private pull requests, contractors unfamiliar with a codebase can be **5–18x slower** than maintainers who know it well. Model performance tracks much closer to contractor-level speed than maintainer-level speed, since the model has no prior context on that specific codebase either — suggesting these models currently behave more like a low-context contractor than a domain expert, even when they're technically capable of solving the underlying problem.

GDPval: Testing Models Against Industry Experts

Paper: arxiv.org/abs/2510.04374

METR asks: *can the model do this task at all, and how long would it take a human?* GDPval asks a different question: *if you handed this task to the model instead of a human professional, is the output actually good enough?* The measurement is a **win rate** — model output vs. an industry expert's output, judged head-to-head.

Task Sourcing

Tasks were built from the real work of industry professionals with an average of **14 years of experience**, spanning a wide range of sectors: real estate, government, manufacturing, professional/scientific/technical services, healthcare, finance, retail, wholesale, and information (including film and video editing). Since models are increasingly multimodal, this broad range makes sense to test — though naturally, models won't be equally strong across all of it.

Example Tasks

A sample of the kinds of tasks included:

- **Manufacturing engineer:** design a 3D model of a cable reel stand for an assembly line.
- **Financial/investment analyst:** create a competitive landscape analysis (a research-heavy task).
- **Registered nurse:** assess images of an issue and create a consultation report.
- **Film/video editor:** create an intro reel from a given script.
- **Customer service:** draft an email response to a dissatisfied customer.
- **Concierge:** create an itinerary for a family of four.
- **Auditor:** identify pricing inconsistencies across several purchase orders.
- **Real estate agent:** design a sales brochure for a new property.
- **Recreation worker:** optimize the layout of a vendor fair.

These are genuinely sourced from real professional work — and many of them are inherently **subjective** to evaluate (there's rarely one single "correct" brochure design or itinerary). That subjectivity is exactly why the benchmark uses a comparative win rate rather than a strict correct/incorrect grading scheme — real-world work often requires both context and subjective judgment to evaluate well.

(Worth separating two different questions here: whether a model **can** produce good output for a task, versus whether it **should** be trusted to do so — e.g., for tasks with real ethical, compliance, or safety stakes. GDPval is measuring the first question, not making a judgment about the second.)

Scope of the Benchmark

- Targets the **top 9 sectors** by contribution to U.S. GDP, covering **44 occupations**.
- **1,320 total tasks**, with **220** released as an open "gold" subset on Hugging Face.
- Predominantly **digital, computer-based tasks** — about 60% of the underlying O*NET occupational taxonomy's computer-based tasks made it into the benchmark, giving fairly representative coverage of what an occupational expert actually spends their time on.

- **Task duration:** averaging up to ~7 hours to complete, with some tasks requiring **weeks**.
- **Modality:** both text-based and multimodal (CAD files, video, audio, spreadsheets, presentations).
- **Task value:** averaging around **\$400 per task** in the gold subset, with some higher-value tasks included too.
- **~70%** of tasks require interacting with a reference file to produce the final output.
- **~89%** of tasks were vetted by experts as being clearly well-specified — reducing the risk that a task's difficulty comes from ambiguity rather than genuine task complexity.

Results: A Different Trend Than METR

Plotting win rate against industry professionals (with more than a decade of experience) across model generations — from GPT-4o up through GPT-5-high and Claude Opus 4.1:

- **GPT-4o (2024):** around **12.4%** win rate.
- Progressing through later models: into the **20s**, then **30s**.
- **Claude Opus 4.1:** reaching **47.6%**.

This trend looks roughly linear over time — notably different from METR's exponential, doubling-every-7-months trend. Since GDPval measures real work compared directly against experts (rather than a duration-based time horizon), it captures reliability and output quality in a way that resists the kind of runaway extrapolation people sometimes assume from an exponential curve (e.g., "if it can do 1 hour now, it'll do a couple of days soon"). Breaking performance down by profession, rather than purely by hours, makes the picture more concrete and realistic: is the model's output actually good enough for this specific kind of real work?

Model-specific strengths noticed: Claude tended to be stronger on aesthetics, document formatting, and understanding PDFs/spreadsheets/presentations, while GPT-5 tended to be stronger on instruction following, correct calculations, and text-based tasks.

Failure Modes

The most common failure category was **instruction following** — models sometimes claim they'll consult reference data provided for a task, but then don't actually use it, substituting hallucinated content instead. Formatting errors were also a recurring issue. GPT-5 showed fewer instruction-following errors than earlier models in this specific evaluation.

How "Good" Is "Good Enough"?

Looking at outcome categories for one model's outputs: roughly **half** of tasks were rated **acceptable but subpar** compared to the human expert's work, about **20%** were rated as

the model genuinely doing **better**, and about **29%** were rated **bad or catastrophic** — output that wasn't acceptable at all. There was also some real disagreement between human graders on how to classify a given output, which adds noise to this breakdown.

Does Repeated Sampling Help Here Too?

Consistent with a broader theme in this notebook series: trying a task multiple times (parallel or sequential sampling, with the model revising based on what it observes) improved results here as well. Comparing a single attempt against multiple attempts with self-correction: GPT-5 saw roughly a **1.6x** improvement in cost-efficiency and about **1.4x** in speed, relative to an unaided human expert.

Even with these gains, in cases where the model succeeds, the cost is still **well under 10%** of the equivalent human expert's salary cost for that work — meaning there's a real subset of professional tasks where models are already positioned to take on substantial parts of the work.

Where Performance Varies

- **By sector:** government, retail, and wholesale tasks showed performance close to parity with human experts (though broad sector-level categories can be less informative than looking at specific tasks directly).
- **By task duration:** models perform noticeably better on shorter tasks (up to a few hours); performance clearly declines as task duration grows.
- **By modality:** as noted above, different models have different relative strengths between multimodal and text-based work.

Occupations Where Models Are Already Near or At Parity

Counter and rental clerks, real estate brokers, shipping/receiving/inventory clerks, buyers and purchasing agents, computer and IT managers, and software developers were all occupations where multiple models approached or matched average human-expert performance. (Note: this is *average* expertise being measured, not the top performer in a field — and for occupations like software developer, the relevant "expertise" was often closer to knowing a specific codebase well, rather than a decade of broad domain experience, which is a different kind of skill than, say, an industrial or mechanical engineer's years of experience.)

Additional strong-performing areas: administrative service managers and compliance officers (government), medical and health service managers, personal financial advisors, customer service representatives, editing tasks broadly, and research-heavy tasks like investigative/detective work and news analysis.

An Important Caveat: Under-Specified Prompts Hurt Performance

When context is deliberately stripped out of a task prompt, model performance drops — a few percentage points lower in win rate, and more strikingly, models genuinely struggle to figure out **what to even work on** without that missing context.

This points to something real about actual work: a lot of professional expertise isn't just "execute a known set of steps correctly" — it's figuring out **what the actual problem is**, what to prioritize, and what context matters in the first place. Models tend to do well once a human has already done that framing work and provided full context — but the framing and prioritization work itself, currently, still tends to fall on the human. This connects directly to the earlier finding that contractors unfamiliar with a codebase are much slower than maintainers who already have context: real work is often **context-heavy**, and this benchmark mainly measures "can a capable person execute this, given full context" — not "can a model figure out what needs doing with no context at all."

What This Means in the Near Term

AI assistance is already showing up in real professional workflows, and it's cost-effective specifically when paired with **human oversight** — humans deciding what needs to be worked on, and models executing against that framing. The benefit varies meaningfully by occupation, and different models have different relative strengths, so occupation-specific model choice matters in practice.

Why Measure This at All?

Before a benchmark like this existed, discussions about AI's economic impact on professions were largely qualitative — general claims that "this will affect a lot of jobs" without a concrete, quantifiable way to say *which* tasks, in *which* professions, are actually affected. GDPval provides a verifiable, task-level taxonomy for that question, rather than relying on speculation.

It's also worth being precise about what "the model outperforms the human" means here: since the model is working from the same prompt and context a human would need to have specified, outperforming a human on a *given, well-specified task* is different from outperforming a human on the *broader job* — which includes figuring out what to prioritize and where to spend effort in the first place, something this evaluation doesn't directly measure.

DeepScholar-Bench: Can AI Do Research Synthesis?

Paper: arxiv.org/abs/2508.20033

Neither METR nor GDPval directly tests a very common knowledge-work skill: **can a model study a set of references, retrieve the relevant information, and synthesize it into a coherent report with verifiable citations?** DeepScholar-Bench (Stanford) targets exactly this — evaluating "deep research" style systems, a space that already has several industry prototypes (OpenAI Deep Research, Gemini Deep Research, Perplexity, Stanford's STORM, OpenScholar, among others).

The Task: Writing a Related Work Section

The benchmark's concrete task is generating the **related work section** of an academic paper — a task nearly every researcher needs, and one that requires genuinely synthesizing prior work, not just answering a single factual question.

Staying Live and Avoiding Contamination

The dataset is built from **recent arXiv papers** (PhD-level difficulty, across 22 domains), and is **rerun monthly** with new papers — so it doesn't go stale, and it specifically avoids testing on papers a model could have seen during training, since only post-training-cutoff papers are used.

Three Evaluation Axes

1. **Knowledge synthesis:** is the output well-organized and coherent, and does it capture the key facts ("nuggets") from the source material — not just readable, but actually complete?
2. **Retrieval quality:** are the references actually relevant to the topic, and does the set of retrieved papers include the important, foundational ones (not just tangentially related ones)?
3. **Verifiability:** do the citations actually support the specific claims being made? This splits into two related but distinct questions — precision (is each individual citation accurate?) and coverage (are all the claims that need a citation actually backed by one?).

Each metric was validated against human judgment, reaching **70–80% agreement** with human evaluators — giving reasonable confidence that the automated scoring reflects genuine quality.

A Genuinely Unsaturated Benchmark

Most benchmarks eventually saturate — once scores hit 70–80%, there's little headroom left to measure further progress. DeepScholar-Bench is different: **no existing system**

exceeds 19% across its combined metrics. This is a benchmark with substantial room left to improve on, not one that's already been "solved."

Results by Axis

- **Knowledge synthesis:** OpenAI Deep Research performed comparatively well here — but across the board, models consistently **miss key facts**, even while writing fluent, coherent English. Good prose doesn't guarantee complete coverage of the source material.
- **Retrieval quality:** generally weaker across all systems — every system struggled to find a comprehensive, genuinely important set of sources; document importance scores stayed under **12.5%** across the board.
- **Verifiability:** varied a lot by system — one reference pipeline (DeepScholar-base) reached up to **90% precision**, while OpenAI Deep Research scored lower on verifiability specifically, despite writing noticeably polished prose. Fluent writing and citation accuracy don't necessarily move together.

Failure Modes

- **Missing foundational sources.** Even when systems find relevant documents, they often miss the truly foundational papers in a field — the ones a genuine domain expert would know by heart from accumulated experience, not just from a single search pass.
- **Weak sense of document importance.** Systems can judge topical relevance reasonably well, but struggle to judge which sources are actually the *most important* ones to include, beyond simple relevance.
- **Missing key facts even with perfect sources.** Even when given the ideal set of source papers directly (removing the retrieval problem entirely), systems extracted only about **50%** of the key facts from those papers — and coverage was lower still when they had to find the right papers themselves.
- **A synthesis-vs-verifiability trade-off.** No system currently excels at both producing high-quality synthesis *and* maintaining high verifiability at the same time — strength in one area didn't reliably carry over to the other.

The broader lesson: referencing external knowledge bases well is a genuinely hard, distinct skill — separate from simply completing a long task (METR) or matching an expert's output on a well-specified task (GDPval). It's not solved just by scaling task duration or improving instruction-following; retrieval quality, factual coverage, and citation accuracy are each their own bottleneck.

Pulling the Three Benchmarks Together

Research Synthesis Sits at the Intersection

Research synthesis tasks typically take **30 minutes to 8 hours** (per METR's framing) — currently around the edge of what models can attempt (roughly 50 minutes at present capability) — but the **quality** at that time horizon can still be poor, often only reaching something like 50% real success. This shows a task can look "in range" on a duration-based benchmark while still falling short on a quality-based one — the two axes genuinely measure different things.

Research synthesis also has clear **economic value** (financial research, competitive analysis, and similar tasks all depend on this skill) — but the capability gaps are concrete: multi-step reasoning, comprehensive information gathering, and maintaining verifiability while combining information pulled from multiple separate tool calls and agent runs (a context-engineering challenge in its own right).

How the Three Benchmarks Differ Structurally

Aspect	METR	GDPval	DeepScholar-Bench
Scoring	Automated	Human-graded, pairwise win rate	Automated, validated against humans
Multi-agent interaction	No — single-agent	Not primarily	Not primarily
Task specification	N/A	Extremely well-specified, all context given upfront	Context must be actively retrieved
Iteration	N/A	One-shot, no back-and-forth correction	N/A
Core demand	Sustained task duration	Tacit/pre-existing model knowledge	Retrieval, synthesis, and verifiability

Each benchmark captures a different piece of what "being good at real-world knowledge work" actually requires — no single one tells the whole story on its own.

Where Model Capability Stands Overall

What Models Are Reliably Good At

- **Isolated, well-specified tasks** in domains the models handle well — software engineering and ML research being the clearest examples, showing strong hour-long-task performance (plausibly because these are domains where model builders themselves have the deepest expertise to draw on).
- **Producing well-organized output**, even when not every part of that output is fully correct.

Where Confidence Is Still Low

- **Tasks requiring a lot of implicit context** — if that context isn't explicitly given in the prompt, models struggle to identify what actually needs to be figured out.
- **Ambiguous, under-specified prompts** in general.
- **Adversarial environments.**
- **True 95%+ reliability** — not yet consistently reached.
- **Generalization meaningfully beyond software and general knowledge work.**
- **Referencing knowledge bases well** — comprehensive high-quality source-finding, surfacing key facts, and verifying claims with accurate citations all remain genuine weak points.

Final Summary Across All Three Benchmarks

- **METR:** time horizon (at 50% reliability) has been **doubling roughly every 7 months** — extrapolated forward, this suggests month-long tasks could become feasible somewhere around 2028–2031.
- **GDPval:** shows a much more **linear** improvement trend — win rate against experts reaching around 48% for top models overall, but with wide variation across task categories, and many categories still well below that.
- **DeepScholar-Bench:** shows that tasks requiring **retrieval and knowledge-base referencing** carry their own distinct, still largely unresolved gaps in quality and verifiability, separate from either raw task duration or expert-level output quality.

The unifying takeaway: a model completing longer and longer tasks does **not** automatically mean the output is reliable or high-quality at that length. Measuring real progress requires multiple complementary lenses — **duration-based**, **economic-value-based**, and **synthesis-quality-based** metrics together — each validated against actual human performance, since no single metric captures the full picture on its own.

Open Questions

Will Coding Continue to Improve Fastest, and What Will Lag?

Coding-related benchmarks (like SWE-bench Verified) have shown particularly strong, consistent growth. But a large category of harder, less narrowly-scoped tasks — beyond simple pull-request-style fixes — remains genuinely difficult. Distributed systems work is one example of a domain that tends to stay hard for models to get right, since it often requires reasoning about complex, hard-to-fully-specify interactions rather than following a clear, bounded task description.

What's Actually Blocking Progress on the Hardest ("Long Tail") Problems?

A mix of two things: **data availability** and **fundamental model capability limits**. Broad domains with abundant, structured data (legal research, financial research) have seen comparatively fast adoption and improvement. Narrower, more specialized domains — where there simply isn't much representative task data, or where the model's underlying capability still falls short — remain much harder to close the gap on. Robotics/embodied AI is one clear example where a lot of current effort is specifically focused on data collection as the primary lever for progress, rather than assuming model architecture alone will solve it.

How Is "50% Success Rate" Actually Computed in METR?

To be precise: for a single task, the model is run **multiple times** (not just once), and the success rate for that task is the fraction of those attempts that succeeded. The overall time-horizon curve is then built by looking across many tasks and their individual success rates, plotted against each task's estimated human-completion time — not a single aggregate "50% of all tasks succeed" statistic, but a per-task attempt-based success rate fitted into an overall curve.